Integration Clinic: project handoff

State as of 4 November 2026. Written so that a person or an LLM can understand the whole project without other context: what it is, how it's built, what was done, and what's left. No secret values appear in this document.

1. What this is

Integration Clinic is a public demo store running on Razorpay **test mode** that breaks on purpose. A visitor can buy something with a test card (the correct flow), then pick one of eight common merchant integration mistakes and run the same action against a deliberately vulnerable route and against the correct route. For each mistake the site shows three panels: the merchant's code, the real request and response log (with Copy as cURL), and the root cause with a fix diff. Each mistake also has a troubleshooting doc page. It's a proof of concept, a debugging showcase and a writing sample in one repo. It's unofficial and not affiliated with Razorpay.

Status: built, deployed and working. Real Razorpay payments and webhooks go through the live site, and real webhook deliveries have been received and processed.

Item	Value
Live site	https://ic-chi-silk.vercel.app
Source code	https://github.com/cipher2150/IC (branch main). Local copy: ~/Desktop/IC
Hosting	Vercel, imported from the GitHub repo. Every push to main redeploys automatically.
Database	Neon Postgres 18.6. Project bold-boat-81666053, branch production, org org-polished-glitter-33931902, AWS us-east-2.
Payments	Razorpay test mode (key id prefix rzp_test_). Webhook registered at https://ic-chi-silk.vercel.app/api/webhooks/razorpay
Stack	Next.js 16.4.4 (App Router), React 19.3, TypeScript, Tailwind CSS 4, razorpay SDK 2.9.8, @neondatabase/serverless 1.2 (HTTP driver, no ORM), MDX via @next/mdx, Vitest 4
Owner	Rajshekhar Prasad Saxena (GitHub cipher2150)

2. What was done, in order

- **Spec.** The owner provided a 13-section spec: catalog, 5 core and 3 optional mistake scenarios, Payment Links, refunds, MDX docs, a 3-table raw-SQL data model, Vercel deployment notes, edge cases, tests, and presentation. Decisions: build the whole spec in one pass, and use npm.
- **Scaffold.** create-next-app 16.4.4. The folder name "IC" breaks npm's lowercase package-name rule, so the app was scaffolded as "integration-clinic" elsewhere and moved in. The scaffold's default cacheComponents flag was turned off, because it forbids the spec's required route config export const runtime = 'nodejs'.
- **Build.** Library code, 13 API routes (including 7 deliberately broken ones), the store UI, scenario pages, 10 MDX doc pages, a Postman collection, a Vitest suite and the README.
- **First verification, with no credentials.** The build, lint, type check and unit tests pass, and the curl checks behave as intended (clear config errors, a 429 rate limit, a 400 for a bad webhook signature, a 404 for an unknown scenario).
- **Razorpay keys.** The owner added test keys. They had been typed into .env.example, a git-tracked file, so they were moved to the git-ignored .env.local and .env.example went back to placeholders, before any commit. The owner chose the webhook secret.
- **Neon.** The neon CLI was installed (v8.17.1). neon init linked the project and created neon.ts, and it also installed Neon agent plugins and skills into the owner's user-level settings for several AI tools. The owner ran neon login, and neon deploy wrote DATABASE_URL, DATABASE_URL_UNPOOLED and NEON_BRANCH into .env.local. psql couldn't connect: port 5432 is reachable, but this network cuts off the Postgres handshake. So db/schema.sql was applied through the HTTPS driver over the direct (unpooled) URL.
- **Local end-to-end run** against real Razorpay test mode and Neon. Every route was exercised and real error responses were captured for the docs. The test rows were deleted afterwards.

- **Hosting.** The owner chose to host everything (no ngrok tunnel). The owner pushed to GitHub and imported the repo into Vercel with four env vars. The live site was verified: pages, the public webhook route, the webhook secret, and a real order created through Vercel into Neon.
- **Webhook and first real payment.** The owner registered the webhook. A real Rs. 499 test payment was captured. /verify and the webhook both marked it paid and it was fulfilled exactly once, which proves the idempotency design. Two earlier attempts failed because an international test card was used.
- **Refinements.** Switched the UI and docs to Indian test cards (the account accepts domestic cards only) and added a test-card table to the README. Captured the real over-refund error. The owner reported that Refresh didn't seem to work. A headless-browser test showed it did refetch but gave no feedback, so Refresh now shows its state and the update time. A silent failure on non-JSON responses and a missing error body for Razorpay 404s were also fixed, and GET /api/orders is now marked private and not cacheable.

3. How it works

Happy path.

```
1. Browser  -> POST /api/orders {sku}
   Server looks up the price, converts to paise, calls razorpay.orders.create,
   inserts the order (status=created), returns {order_id, amount, key_id}.
2. Browser  -> Razorpay Checkout.js
   Returns {razorpay_order_id, razorpay_payment_id, razorpay_signature}.
3. Browser  -> POST /api/verify
   HMAC-SHA256(order_id|payment_id, key_secret) with a timing-safe compare, then
   payments.fetch: captured + same order id + same amount -> paid.
4. Razorpay -> POST /api/webhooks/razorpay
   Raw body (req.text()), verify X-Razorpay-Signature with the webhook secret,
   insert X-Razorpay-Event-Id (unique), apply once, always return 200.
```

- **The webhook is the source of truth.** /verify is only fast feedback for the buyer. Both call one function, advanceOrder() in lib/db.ts, which is the only place order status changes.
- **Status only moves forward:** created -> failed -> paid -> refunded. A single UPDATE refuses backward moves, so /verify and the webhook can arrive in either order. fulfilment_count goes up by 1 in the same UPDATE as the move to paid, so an order is fulfilled exactly once.
- **Webhook dedupe:** the event id is inserted into webhook_events (primary key) before the event is applied. Duplicates return 200, because any 4xx or 5xx makes Razorpay retry. If applying fails, the event row is deleted and the route returns 500, so a retry gets through.
- **Sessions:** proxy.ts (the Next 16 name for middleware) gives each visitor a random ic_session cookie, and all orders are scoped to it. The cookie is deliberately not httpOnly, so Copy as cURL can include it. It's a demo-scoped random id, not a login.
- **Rate limit:** proxy.ts allows 20 POSTs per minute per IP on /api/*, and webhooks are exempt. It's in-memory per serverless instance.
- **Payment Links** reuse the orders table (the plink_ id is stored in rzp_order_id), and payment_link.paid moves them to paid. **Refunds** call payments.refund, and the status becomes refunded only when refund.processed arrives.
- **Key guard:** getRazorpay() refuses any key id that doesn't start with rzp_test_, or a missing secret, before any API call. Clients are created lazily, so next build works without env vars.

4. File map

Path	Purpose
app/page.tsx	Store: catalog of 3 items (Buy, Pay via link), scenario picker, Your orders
app/components/BuyButton.tsx	Order, then Checkout, then /verify. Also the Payment Link action.
app/components/Orders.tsx	Session's orders, Refresh (with state and time), Refund button
app/components/ScenarioRunner.tsx	Broken/Fixed toggle, the per-scenario request flows (a switch on slug), the request log, Copy as cURL

Path	Purpose
app/scenarios/[slug]/page.tsx	Three panels: merchant code, what happened, root cause and fix diff
app/docs/*/page.mdx	10 docs: 8 troubleshooting pages + Checkout in 10 minutes + Go-live checklist
app/api/orders | verify | refunds | payment-links	Correct flows (GET /api/orders lists the session's orders)
app/api/webhooks/razorpay/route.ts	Webhook handler (source of truth)
app/api/broken/*	Deliberately vulnerable routes: amount, confirm, client-order, keys, webhook, webhook-parsed, refund
app/api/scenarios/replay-webhook	Signs a synthetic payment.captured event for a fresh demo_order_ row (scenarios #5 and #6)
lib/razorpay.ts	Key guard, getRazorpay, sign/verifySignature (timing-safe), fail, errorResponse
lib/db.ts	Neon HTTP client, session helper, order queries, advanceOrder, processEvent (dedupe + apply)
lib/catalog.ts	CATALOG, getItem, toPaise = Math.round(rupees * 100), formatPaise
lib/scenarios.ts	Scenario registry (title, symptom, cause, fix, routes, snippet, diff) and the guides list
lib/checkout.ts	Browser helpers: openCheckout, postJson, isRealPayment
proxy.ts	Session cookie + rate limit
db/schema.sql	The schema (idempotent CREATE ... IF NOT EXISTS)
tests/clinic.test.ts	Vitest: HMAC, toPaise, webhook dedupe against the real DB
public/integration-clinic.postman_collection.json	Postman collection for the correct flow
neon.ts, .neon	Neon CLI config and project link (.neon is git-ignored)
AGENTS.md	Written by Next.js: read node_modules/next/dist/docs before coding

5. Data model (raw SQL, no ORM)

```
orders(id bigserial PK, session_id text, sku text, amount_paise int > 0,
       rzp_order_id text UNIQUE NOT NULL   -- order_..., plink_..., or demo_order_... (synthetic)
       rzp_payment_id text, status text CHECK IN ('created','failed','paid','refunded'),
       fulfilment_count int default 0, created_at timestamptz)
webhook_events(event_id text PK, type text, payload jsonb, received_at timestamptz)
```

The spec said "3 tables" but described only these two. That's all the app needs.

6. The eight scenarios

#	Mistake	Broken route	What you see (broken)	Fixed route / fix
1	Rupees instead of paise	/api/broken/amount	Rs. 4.99 order for a Rs. 499 item; real 400 for 499.5	/api/orders; toPaise()
2	Skipping server verification	/api/broken/confirm	Order paid with a fake payment id	/api/verify; HMAC + payments.fetch
3	Trusting the client price	/api/broken/client-order	Rs. 1 order for a Rs. 999 item	/api/orders; client sends only sku
4	Wrong or mixed keys	/api/broken/keys	Real 401 Authentication failed	Key guard + env vars
5	Non-idempotent webhook	/api/broken/webhook	fulfilment_count = 2 after replay	Event-id dedupe + forward-only
6	Parse before verify	/api/broken/webhook-parsed	Signature mismatch	req.text(), verify, then parse
7	Over-refund	/api/broken/refund	Real 400 from refunds API	/api/refunds validates first

#	Mistake	Broken route	What you see (broken)	Fixed route / fix
8	Modal closed, order stuck	Checkout without handlers	Spinner forever, order stays created	ondismiss + payment.failed

Adding a scenario takes one entry in lib/scenarios.ts, one route, one case in ScenarioRunner, and one MDX page. Broken routes are intentionally wrong. Don't "fix" them.

7. Environment variables

Name	Used by	Where it's set
RAZORPAY_KEY_ID	Razorpay API, also sent to the browser for Checkout	.env.local and Vercel
RAZORPAY_KEY_SECRET	Razorpay API, /verify HMAC	.env.local and Vercel (secret)
RAZORPAY_WEBHOOK_SECRET	Webhook signature check, replay signing	.env.local and Vercel (secret); must match the Razorpay dashboard
DATABASE_URL	App queries (Neon pooled URL)	.env.local and Vercel (secret)
DATABASE_URL_UNPOOLED, NEON_BRANCH	Schema changes only (direct URL); not used by the app	.env.local only (written by neon deploy)

Real values are only in .env.local (git-ignored) and the Vercel project settings. .env.example holds placeholders. The spec's NEXT_PUBLIC_RAZORPAY_KEY_ID was dropped on purpose, because /api/orders already returns key_id.

8. Decisions and deviations from the spec

- cacheComponents is **off** in next.config.ts. With it on, Next 16.4 rejects export const runtime = 'nodejs', which the spec requires on every Razorpay route.
- The key guard runs on first use rather than at boot, so builds and local dev work before keys exist. Prefix rule: rzp_test_ (the app is test-mode only).
- Signature verification is hand-rolled (crypto.timingSafeEqual with a length check first), because the SDK helper uses a plain === compare.
- Fulfilment is tied to the status transition, and event-id dedupe is layered on top, as the spec asked.
- Docs use lists, not tables, to avoid adding a remark-gfm dependency. The scenario diff panel is a plain pre block with +/- line colours, with no syntax highlighter.
- Scenario #7 needs a real captured payment, so it works only after a real test purchase in the same browser session.
- Synthetic ids: demo_order_, demo_pay_ and demo_evt_ (webhook replays), and pay_FAKE (scenario #2). The Refund button and #7 ignore these.

9. Evidence that it works

- npm run build passes (36 routes and pages), lint is clean, the type check is clean, and npm test passes 10 of 10, including the webhook dedupe test against the real Neon DB.
- Local and live end-to-end runs: real Razorpay orders created; double-click reuses the open order; a forged signature gets 400; a signed webhook applies once and its duplicate gets 200; a bad signature gets 400; an over-amount refund is blocked before the API; all 8 broken/fixed pairs behave as designed; a real Payment Link short URL is created.
- Live webhook deliveries from Razorpay were received: payment.authorized, payment.captured, and 2 payment.failed. One real Rs. 499 payment was captured, marked paid, and fulfilled once.
- Headless Chrome test of the live UI: Refresh refetches GET /api/orders and the row updated from created to paid.

10. Real API responses captured

Scenario	Request	Real response
#1	orders.create amount 499.5	400 BAD_REQUEST_ERROR: "The amount must be an integer." (reason input_validation_failed)
#4	wrong secret / placeholder rzp_live_ key	401 BAD_REQUEST_ERROR: "Authentication failed"
#7	refund 50000 on a 49900 captured payment	400 BAD_REQUEST_ERROR: "The refund amount provided is greater than amount captured"
Cards	International test card on this account	payment.failed: international_transaction_not_allowed (domestic cards only)

11. Open items and next steps

- **Push the latest local changes** (not yet committed when this was written): the Refresh/Refund feedback, error-message fix, orders cache header, Indian test cards in the UI, docs and README, and the captured #7 response. Files: README.md, app/page.tsx, app/components/Orders.tsx, lib/checkout.ts, lib/razorpay.ts, app/api/orders/route.ts, app/docs/checkout-in-10-minutes/page.mdx, app/docs/over-refund/page.mdx. Vercel redeploys on push.
- **Check the Razorpay webhook events.** It must include payment.captured, payment.failed, payment_link.paid and refund.processed. No refund.processed or payment_link.paid delivery has been seen yet, so a refunded order stays paid until refund.processed is ticked.
- Test a full refund and a paid Payment Link on the live site. Then confirm the payload paths for payment_link.paid (payment_link.entity.id) and refund.processed (refund.entity.payment_id) against the first stored webhook_events.payload.
- Record the 60-second GIF and the 2-minute demo video for the README (the GIF placeholder is docs/demo.gif).
- Optional hardening: regenerate the Razorpay test key (the secret appeared in a chat), use a long random webhook secret, and move the rate limit to a shared store (Upstash or Vercel WAF).
- Not built (stretch items in the spec): live webhook log panel (Hookscope), "download this scenario as a failing test", and a partial-refund UI.

12. How to work on this project

```
npm install                 # Node 20.12+
npm run dev                 # http://localhost:3000 (needs .env.local)
npm test                    # Vitest; DB test runs when DATABASE_URL is set
npm run build && npm run lint
neon env pull               # refresh Neon env vars into .env.local (needs neon login)
git push                    # Vercel redeploys main automatically
```

- Read node_modules/next/dist/docs before using Next.js APIs (AGENTS.md says so). This Next version differs from older ones: proxy.ts replaces the deprecated middleware.ts, and page and route params are Promises.
- Keep cacheComponents off while routes export runtime = 'nodejs'.
- All order status changes go through advanceOrder(). Fixed routes never read an amount from the client.
- Apply schema changes through the Neon SQL editor or the HTTPS driver with DATABASE_URL_UNPOOLED. psql on port 5432 was blocked on the owner's network.
- Test with Indian cards, for example Visa 4100 2800 0000 1007 or Mastercard 5500 6700 0000 1002, with any future expiry, any CVV, and any 4–10 digit OTP.
- Never commit .env.local. Never log or return the key secret or the webhook secret.